Teoría de grafos y algoritmos

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

3 de marzo de 2026

Graph Theory and Algorithms

En este capítulo se consideran los conceptos fundamentales de la teoría de grafos, la representación de grafos por medio de estructuras de datos y algunos de los algoritmos más importantes para resolver problemas de grafos.

Universidad Pontificia Bolivariana

Índice

1. Fundamentos de teoría de grafos (Graph Theory) 3
1.1. Introducción a los Grafos . . . 3
1.2. Modelado de Situaciones Prácticas con Grafos . . . 3
1.3. El Grafo como Abstracción General . . . 4
1.4. Tipos de Grafos . . . 4
1.4.1. Grafos No Dirigidos . . . 4
1.4.2. Grafos Dirigidos . . . 4
1.5. Representación por Conjuntos . . . 4
1.5.1. Grafo No Dirigido . . . 4
1.5.2. Grafo Dirigido . . . 5
1.5.3. Grafos con Peso o Ponderados . . . 5
1.6. Terminología . . . 5
1.7. ADTs de Grafos . . . 8
1.7.1. Abstract Data Type (ADT) Grafo No Dirigido . . . 8
1.7.2. ADT Grafo Dirigido . . . 8
1.8. Representación de Grafos . . . 8
1.8.1. Matrices de Adyacencia . . . 8
1.8.2. Listas de Adyacencia . . . 9
1.9. Bibliotecas para representación y manipulación de grafos . . . 9
1.10. Algoritmos Fundamentales de Grafos . . . 9

2. Algoritmos de recorrido de grafos 12
2.1. Aplicaciones de los Recorridos de Grafos . . . 12
2.2. Recorrido en profundidad (DFS) . . . 12
2.2.1. Características de DFS . . . 13
2.2.2. Ejemplo Visual de DFS . . . 13
2.2.3. Pseudo-código para DFS . . . 13
2.2.4. Complejidad de DFS . . . 13
2.3. Recorrido en anchura (BFS) . . . 14
2.3.1. Características de BFS . . . 14
2.3.2. Ejemplo de BFS . . . 14
2.3.3. Pseudo-código para BFS . . . 14
2.3.4. Complejidad de BFS . . . 15
2.4. Comparación entre DFS y BFS . . . 15
2.5. Ordenamiento topológico (Topological Sort) . . . 15
2.5.1. Ejemplo de DAG con Dependencias . . . 15
2.5.2. Ordenamiento topológico de Tareas . . . 15
2.5.3. Aplicaciones del Ordenamiento Topológico . . . 16
2.5.4. Pseudo-código para el Ordenamiento Topológico . . . 16
2.5.5. Complejidad del Ordenamiento Topológico . . . 17

3. Caminos más cortos (Shortest paths) 19
3.1. Grafos con Peso o Costo . . . 19
3.2. El Problema del Camino Más Corto . . . 19
3.3. Algoritmo de Dijkstra . . . 19
3.3.1. Pseudo-código del Algoritmo de Dijkstra . . . 20

3.3.2. Ejemplo de Ejecución del Algoritmo de Dijkstra . . . 20
3.4. Eficiencia del algoritmo de Dijkstra . . . 20
3.5. Algoritmo de Bellman-Ford . . . 22
3.5.1. Detección de Ciclos Negativos . . . 22
3.5.2. Pseudo-código del Algoritmo de Bellman-Ford . . . 22
3.5.3. Complejidad de Bellman-Ford . . . 23
3.6. Algoritmo de Floyd-Warshall . . . 23
3.6.1. Pseudo-código del Algoritmo de Floyd-Warshall . . . 24
3.6.2. Complejidad de Floyd-Warshall . . . 24
3.7. Comparación de Algoritmos de Caminos Más Cortos . . . 24
3.8. Algoritmo A* . . . 26
3.8.1. Función Heurística . . . 26
3.8.2. Pseudo-código del Algoritmo A* . . . 27
3.8.3. Ejemplo de Ejecución del Algoritmo A* . . . 27
3.8.4. Complejidad y Optimalidad . . . 29

4. Árboles de Cubrimiento Mínimo (Minimum Spanning Trees) 30
4.1. Algoritmo de Kruskal . . . 30
4.1.1. Pseudo-código del Algoritmo de Kruskal . . . 31
4.1.2. Complejidad del Algoritmo de Kruskal . . . 31
4.2. Algoritmo de Prim . . . 31
4.2.1. Pseudo-código del Algoritmo de Prim . . . 32
4.2.2. Complejidad del Algoritmo de Prim . . . 32
4.3. Comparación de Algoritmos de MST . . . 32

5. Flujo Máximo / Corte mínimo 34
5.1. Flujo Máximo (Maximum Flow) . . . 34
5.2. Algoritmo de Ford-Fulkerson . . . 34
5.2.1. Pseudo-código del Algoritmo de Ford-Fulkerson . . . 35
5.2.2. Complejidad del Algoritmo de Ford-Fulkerson . . . 35
5.3. Corte Mínimo (Minimum Cut) . . . 35
5.3.1. Teorema de Flujo Máximo - Corte Mínimo . . . 36
5.3.2. Aplicaciones del Corte Mínimo . . . 36

1. Fundamentos de teoría de grafos (Graph Theory)

La teoría de grafos es fundamental en ciencias de la computación, ya que proporciona una poderosa herramienta para modelar y resolver problemas que involucran relaciones entre objetos. En este capítulo, aprenderemos los conceptos básicos de grafos, sus diferentes tipos, representaciones y algoritmos fundamentales que nos permitirán resolver problemas prácticos de manera eficiente.

1.1. Introducción a los Grafos

Un grafo es una estructura matemática utilizada para representar relaciones entre objetos. Formalmente, un grafo G se define como un par $G = (V, E)$, donde V es un conjunto de **vértices** (o nodos) y E es un conjunto de **aristas** (o enlaces) que conectan pares de vértices.

1.2. Modelado de Situaciones Prácticas con Grafos

Los grafos son herramientas poderosas para modelar una amplia variedad de situaciones prácticas. A continuación, se presentan algunos ejemplos comunes:

- **Redes sociales**:
  - Vértices: Personas
  - Aristas: Relaciones 'ser amigo', 'ser seguidor'
  - Aplicación: Identificar comunidades (intereses), sistemas de recomendación (publicidad)
- **Redes de transporte**:
  - Vértices: Ciudades, estaciones, aeropuertos, etc.
  - Aristas: Carreteras, rutas aéreas o conexiones de tren
  - Aplicación: Encontrar la ruta más corta, optimizar rutas de entrega
- **Redes de computadoras**:
  - Vértices: Dispositivos (routers, switches, computadoras)
  - Aristas: Conexiones de red (físicas o inalámbricas)
  - Aplicación: Optimizar el enrutamiento de paquetes, ruta más corta
- **Circuitos electrónicos**:
  - Vértices: Componentes (resistencias, capacitores, transistores)
  - Aristas: Conexiones eléctricas
  - Aplicación: Análisis de circuitos, diseño de circuitos impresos
- **Planificación de proyectos**:
  - Vértices: Tareas
  - Aristas: Dependencias entre tareas
  - Aplicación: Encontrar la ruta crítica, optimizar el tiempo de ejecución
- **Máquinas de estados finitos**:

- Vértices: Estados
- Aristas: Transiciones entre estados (eventos)
- Aplicación: Modelado de protocolos de comunicación, análisis de léxico en compiladores, diseño de sistemas de control, reconocimiento de patrones y validación de entradas.

1.3. El Grafo como Abstracción General

La belleza de los grafos radica en su capacidad para abstraer la complejidad de un sistema y enfocarse en las relaciones esenciales entre sus componentes. Al ignorar los detalles irrelevantes, los grafos permiten analizar y resolver problemas de manera eficiente.

1.4. Tipos de Grafos

Los grafos se pueden clasificar según diferentes características. A continuación se presentan las clasificaciones más importantes:

1.4.1. Grafos No Dirigidos

En un grafo no dirigido, las aristas no tienen dirección. Es decir, si existe una arista entre los vértices u y v, se puede recorrer en ambas direcciones. Matemáticamente, una arista en un grafo no dirigido es un par no ordenado $\{u, v\}$.

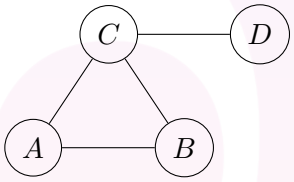


Figura 1: Grafo no dirigido

1.4.2. Grafos Dirigidos

En un grafo dirigido, las aristas tienen dirección. Es decir, si existe una arista desde el vértice u hasta el vértice v, solo se puede recorrer en esa dirección. Matemáticamente, una arista en un grafo dirigido es un par ordenado (u, v).

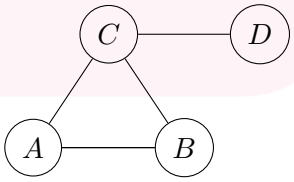


Figura 2: Grafo dirigido

1.5. Representación por Conjuntos

1.5.1. Grafo No Dirigido

Un grafo no dirigido G se define como $G = (V, E)$, donde:

- V es el conjunto de vértices.
- E es el conjunto de aristas, donde cada arista es un par no ordenado de vértices $\{u, v\}$, con $u, v \in V$.

1.5.2. Grafo Dirigido

Un grafo dirigido G se define como $G = (V, E)$, donde:

- V es el conjunto de vértices.
- E es el conjunto de aristas, donde cada arista es un par ordenado de vértices (u, v), con $u, v \in V$.

1.5.3. Grafos con Peso o Ponderados

En muchas aplicaciones prácticas, las aristas de un grafo tienen asociado un valor numérico llamado *peso* o *costo*. Este peso puede representar distancia, costo, tiempo, capacidad, etc. Un grafo con peso se denota como $G = (V, E, w)$, donde $w : E \to \mathbb{R}$ es una función que asigna un peso a cada arista.

Los grafos ponderados son fundamentales para problemas como:

- Caminos más cortos (Dijkstra, Bellman-Ford, Floyd-Warshall)
- Árboles de cubrimiento mínimo (Kruskal, Prim)
- Flujo máximo en redes (Ford-Fulkerson)

1.6. Terminología

- **Camino**: Una secuencia de vértices conectados por aristas.
- **Camino Simple**: Un camino donde no se repiten vértices.

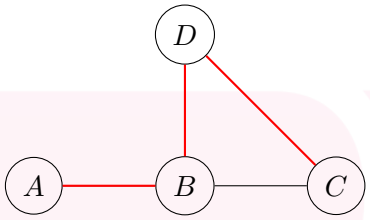


Figura 3: Ejemplo de camino simple: A-B-D-C

- **Ciclo**: Un camino que comienza y termina en el mismo vértice.
- **Ciclo Simple**: Un ciclo donde no se repiten vértices intermedios.
- **Grafo Conexo (en grafos no dirigidos)**: Un grafo no dirigido donde existe un camino entre todo par de vértices.
- **Grafo Débilmente Conexo (en grafos dirigidos)**: Un grafo dirigido es débilmente conexo si al reemplazar todas sus aristas dirigidas por aristas no dirigidas se obtiene un grafo conexo.

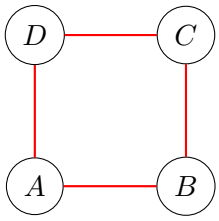


Figura 4: Ejemplo de ciclo simple: A-B-C-D-A

Figura 5: Ejemplo de grafo conexo (izquierda) y no conexo (derecha)

- **Grafo Fuertemente Conexo**: Un grafo dirigido donde existe un camino entre todo par de vértices respetando el sentido de las aristas.

Figura 6: Ejemplo de grafo fuertemente conexo (izquierda) y no fuertemente conexo (derecha)

- **Grafo Acíclico**: Un grafo que no contiene ciclos.

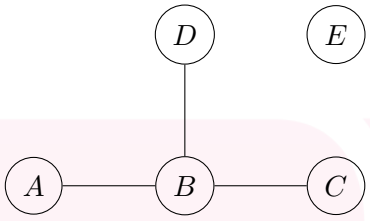


Figura 7: Ejemplo de grafo acíclico

- **Árbol**: Un grafo conexo acíclico.
- **Grafo Acíclico Dirigido (Directed Acyclic Graph (DAG))**: Un grafo dirigido que no contiene ciclos.
- **Grado de un Nodo**: El número de aristas incidentes a un nodo. En grafos dirigidos, se distingue entre grado de entrada (aristas que llegan al nodo) y grado de salida (aristas que salen del nodo).
- **Clique**: Un grafo (o subgrafo de un grafo mayor) en el que todo par de vértices está conectado por una arista.

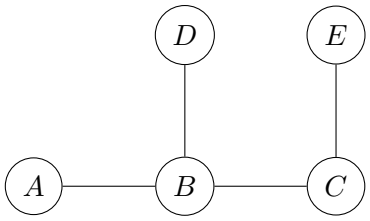


Figura 8: Ejemplo de árbol

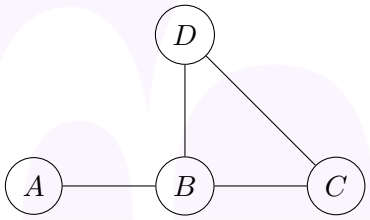


Figura 9: Ejemplo de DAG

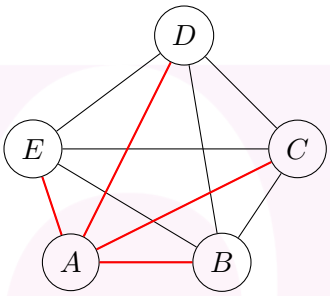


Figura 10: Ejemplo de clique de tamaño 5

Teorema 1 (Cota superior del número de aristas) *El máximo número de aristas de un grafo es $O(V^2)$, siendo el clique el máximo posible.*

- ***Grafo no dirigido:*** *En todo grafo no dirigido con V vértices, el número de aristas E satisface la desigualdad:*
$$E \leq \frac{(V-1)V}{2}$$
- ***Grafo dirigido:*** *En todo grafo dirigido con V vértices, el número de aristas E satisface la desigualdad:*
$$E \leq (V-1)V$$

- **Densidad de un Grafo**: La proporción de aristas presentes en un grafo en relación con el número máximo posible de aristas. Si el número de aristas en el grafo es pequeño con respecto al máximo posible, se dice que el grafo es *disperso* o *no denso*. Un criterio práctico para el análisis de algoritmos es considerar como dispersos los grafos donde $E \leq cV$ para alguna constante c, es decir, cuando el número de aristas es lineal con respecto al número de vértices.

- **Grafo Bipartito**: Un grafo cuyos vértices pueden dividirse en dos conjuntos disjuntos de tal manera que cada arista conecta un vértice de un conjunto con un vértice del otro conjunto. No se permiten aristas entre vértices del mismo conjunto.

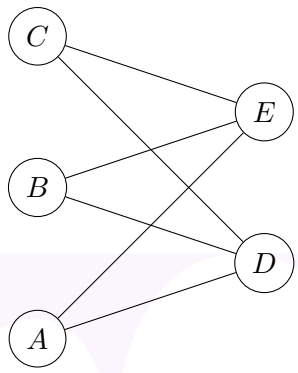


Figura 11: Ejemplo de grafo bipartito

Aplicaciones de grafos bipartitos: Se utilizan frecuentemente para modelar relaciones de asignación de recursos. Por ejemplo la asignación de estudiantes a cursos, o de empleados a tareas.

1.7. ADTs de Grafos

Para definir el ADT de los grafos dirigidos y no dirigidos, vamos a asumir sin pérdida de generalidad que los vértices del grafo se numeran $0 \ldots V - 1$.

1.7.1. ADT Grafo No Dirigido

Método	**Descripción**
`int V()`	Retorna número de vértices
`int E()`	Retorna número de aristas
`void addEdge(int u, int v)`	Agrega la arista $\{u, v\}$
`Iterable<Integer>adj(int v)`	Adyacentes de v
`int degree(int v)`	Retorna el grado de v
`String toString()`	Retorna la representación textual del grafo

1.7.2. ADT Grafo Dirigido

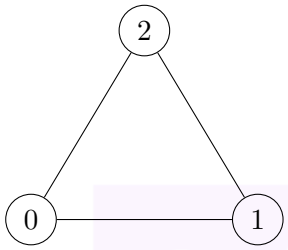
Método	**Descripción**
`int V()`	Retorna número de vértices
`int E()`	Retorna número de aristas
`void addEdge(int u, int v)`	Agrega la arista (u, v)
`Iterable<Integer>adj(int v)`	Adyacentes de v
`int indegree(int v)`	Retorna el grado entrante de v
`int outdegree(int v)`	Retorna el grado saliente de v
`String toString()`	Retorna la representación textual del grafo
`DiGraph reverse()`	Retorna el grafo reverso

1.8. Representación de Grafos

1.8.1. Matrices de Adyacencia

Una matriz de adyacencia es una matriz cuadrada de tamaño $|V| \times |V|$, donde $|V|$ es el número de vértices en el grafo. El elemento (i, j) de la matriz es 1 si existe una arista entre el vértice i y el

vértice j, y 0 en caso contrario.

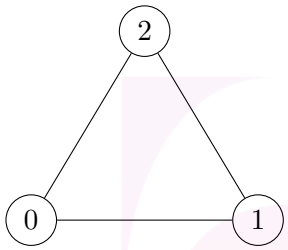


	0	1	2
0	0	1	1
1	1	0	1
2	1	1	0

Figura 12: Ejemplo de grafo y su matriz de adyacencia

1.8.2. Listas de Adyacencia

Una lista de adyacencia es una lista donde cada elemento representa un vértice en el grafo. Cada elemento de la lista contiene una lista de los vértices adyacentes a ese vértice.



Vértice	Adyacentes
0	1, 2
1	0, 2
2	0, 1

Figura 13: Ejemplo de grafo y su lista de adyacencia

1.9. Bibliotecas para representación y manipulación de grafos

Algunos ejemplos de herramientas disponibles para distintos lenguajes:

- **Python**: NetworkX, PyGraphviz, GraphViz.
- **Java**: JUNG, JGraphT, Guava, Apache commons graph.
- **C#**: QuickGraph.
- **C++**: Boost Graph Library.

1.10. Algoritmos Fundamentales de Grafos

Los algoritmos de grafos son herramientas esenciales para resolver problemas prácticos. A continuación, se presentan algunos de los algoritmos más importantes:

- **Recorrido en Profundidad (DFS)**:
  - Propósito: Explorar todos los vértices de un grafo siguiendo un camino hasta el final antes de retroceder.
  - Aplicaciones: Detección de ciclos, ordenamiento topológico, componentes conexas.

  - Complejidad: $O(|V| + |E|)$, donde $|V|$ es el número de vértices y $|E|$ es el número de aristas.

- **Recorrido en Anchura (BFS)**:
  - Propósito: Explorar todos los vértices nivel por nivel, comenzando desde un vértice inicial.
  - Aplicaciones: Encontrar el camino más corto en grafos no ponderados, verificar si un grafo es bipartito.
  - Complejidad: $O(|V| + |E|)$.
- **Ordenamiento Topológico**:
  - Propósito: Ordenar los vértices de un grafo dirigido acíclico (DAG) de tal manera que para cada arista dirigida (u, v), el vértice u aparece antes que el vértice v en el ordenamiento.
  - Aplicaciones: Planificación de tareas, análisis de dependencias.
  - Complejidad: $O(|V| + |E|)$.
- **Algoritmo de Dijkstra**:
  - Propósito: Encontrar el camino más corto desde un vértice origen a todos los demás vértices en un grafo con pesos no negativos.
  - Aplicaciones: Enrutamiento en redes, sistemas de navegación GPS.
  - Complejidad: $O((|V| + |E|) \log |V|)$ usando una cola de prioridad.
- **Algoritmo de Bellman-Ford**:
  - Propósito: Encontrar el camino más corto desde un vértice origen a todos los demás vértices, permitiendo pesos negativos.
  - Aplicaciones: Detección de ciclos negativos, enrutamiento en redes con costos negativos.
  - Complejidad: $O(|V||E|)$.
- **Algoritmo de Floyd-Warshall**:
  - Propósito: Encontrar los caminos más cortos entre todos los pares de vértices.
  - Aplicaciones: Análisis de redes, cálculo de distancias entre todas las ciudades.
  - Complejidad: $O(|V|^3)$.
- **Algoritmo de Kruskal**:
  - Propósito: Encontrar el árbol de expansión mínima de un grafo.
  - Aplicaciones: Diseño de redes de comunicación, planificación de rutas de cableado.
  - Complejidad: $O(|E| \log |E|)$.
- **Algoritmo de Prim**:
  - Propósito: Encontrar el árbol de expansión mínima de un grafo.
  - Aplicaciones: Similar a Kruskal, pero puede ser más eficiente en grafos densos.

- Complejidad: $O((|V| + |E|) \log |V|)$ usando una cola de prioridad.

- **Algoritmo de Ford-Fulkerson**:
  - Propósito: Encontrar el flujo máximo en una red de flujo.
  - Aplicaciones: Optimización de redes de distribución, asignación de recursos.
  - Complejidad: $O(|E|f^*)$, donde f^* es el valor del flujo máximo.

2. Algoritmos de recorrido de grafos

Los algoritmos de recorrido de grafos son fundamentales para explorar y analizar la estructura de un grafo. Permiten visitar cada vértice del grafo de manera sistemática, lo que es esencial para resolver una variedad de problemas, como la búsqueda de caminos, la detección de ciclos y la identificación de componentes conexas.

2.1. Aplicaciones de los Recorridos de Grafos

Los recorridos de grafos tienen numerosas aplicaciones en informática y otras disciplinas, incluyendo:

- **Búsqueda de caminos**:
  - Encontrar la ruta más corta entre dos puntos en un mapa.
  - Planificar rutas de entrega optimizando tiempo y distancia.
  - Navegación GPS y sistemas de recomendación de rutas.
- **Detección de ciclos**:
  - Verificar la integridad de datos en bases de datos.
  - Detectar dependencias circulares en código fuente.
  - Validar la estructura de redes de comunicación.
- **Análisis de redes sociales**:
  - Identificar comunidades y grupos de usuarios.
  - Recomendar conexiones basadas en intereses comunes.
  - Analizar patrones de interacción y difusión de información.
- **Planificación de tareas**:
  - Determinar el orden óptimo de ejecución de tareas.
  - Identificar dependencias críticas en proyectos.
  - Optimizar la asignación de recursos.
- **Resolución de laberintos**:
  - Encontrar la salida más corta en un laberinto.
  - Generar soluciones para puzzles y juegos.
  - Diseñar algoritmos de navegación para robots.

2.2. Recorrido en profundidad (DFS)

El recorrido en profundidad (Depth-first Search (DFS), por sus siglas en inglés Depth-First Search) es un algoritmo para recorrer un grafo que explora tan lejos como sea posible a lo largo de cada rama antes de retroceder. Una analogía útil es imaginar que estás explorando un laberinto: sigues un camino hasta llegar a un callejón sin salida (o un nodo ya visitado), y luego retrocedes para explorar caminos alternativos que no has visitado.

2.2.1. Características de DFS

- Explora un camino completo antes de retroceder.
- Utiliza una estructura de pila (stack) implícita o explícita.
- Es útil para detectar ciclos y componentes conexas.
- Puede implementarse de forma recursiva o iterativa.

2.2.2. Ejemplo Visual de DFS

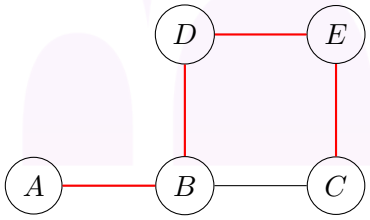


Figura 14: Ejemplo de recorrido DFS: A-B-D-E-C

2.2.3. Pseudo-código para DFS

Algorithm 1 DFS(grafo, vértice)

```
procedure DFS(grafo, vértice)
    Marcar vértice como visitado
    for cada vecino del vértice en el grafo do
        if vecino no está visitado then
            DFS(grafo, vecino)
        end if
    end for
end procedure
```

2.2.4. Complejidad de DFS

La complejidad temporal de DFS es $O(V + E)$, donde V es el número de vértices y E es el número de aristas en el grafo. Esto se debe a que:

- Cada vértice se visita exactamente una vez: $O(V)$
- Cada arista se examina exactamente una vez (dos veces en grafos no dirigidos, una desde cada extremo): $O(E)$

La complejidad espacial de DFS es $O(V)$ en el peor de los casos, debido a:

- La pila de recursión puede crecer hasta V en profundidad (en el peor caso, un camino lineal)
- El arreglo de vértices visitados requiere $O(V)$ espacio

2.3. Recorrido en anchura (BFS)

El recorrido en anchura (Breadth-first Search (BFS), por sus siglas en inglés Breadth-First Search) es un algoritmo para recorrer un grafo que explora todos los vecinos del vértice actual antes de pasar a los vecinos de los vecinos. Es como expandir una onda desde un punto: primero visitas todos los nodos a distancia 1, luego todos los nodos a distancia 2, y así sucesivamente. Esta propiedad hace que BFS sea ideal para encontrar el camino más corto en grafos no ponderados.

2.3.1. Características de BFS

- Explora todos los vértices a una distancia k antes de pasar a distancia k+1.
- Utiliza una estructura de cola (queue).
- Encuentra el camino más corto en grafos no ponderados.
- Es útil para verificar si un grafo es bipartito.

2.3.2. Ejemplo de BFS

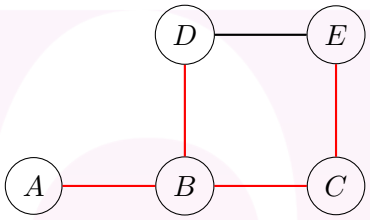


Figura 15: Ejemplo de recorrido BFS: A, B, C-D, E

2.3.3. Pseudo-código para BFS

Algorithm 2 BFS(grafo, vértice)

```
procedure BFS(grafo, vértice)
    Crear una cola Q
    Encolar vértice en Q
    Marcar vértice como visitado
    while Q no está vacía do
        vértice = desencolar de Q
        for cada vecino del vértice en el grafo do
            if vecino no está visitado then
                Encolar vecino en Q
                Marcar vecino como visitado
            end if
        end for
    end while
end procedure
```

2.3.4. Complejidad de BFS

La complejidad temporal de BFS es $O(V + E)$, donde V es el número de vértices y E es el número de aristas en el grafo. Esto se debe a que:

- Cada vértice se encola y desencola exactamente una vez: $O(V)$
- Para cada vértice, se examinan todas sus aristas adyacentes: $O(E)$ en total

La complejidad espacial de BFS es $O(V)$, ya que en el peor de los casos:

- La cola puede contener todos los vértices de un nivel, que podría ser $O(V)$ (por ejemplo, en un grafo estrella)
- El arreglo de vértices visitados requiere $O(V)$ espacio

2.4. Comparación entre DFS y BFS

Aunque DFS y BFS tienen la misma complejidad asintótica $O(V + E)$, tienen características y aplicaciones diferentes:

Característica	**DFS**	**BFS**
Estructura de datos	Pila (recursión)	Cola
Estrategia	Profundidad primero	Anchura primero
Camino más corto	No garantiza	Sí (grafos no ponderados)
Uso de memoria	$O(h)$ donde h es la altura	$O(w)$ donde w es el ancho
Aplicaciones típicas	Detección de ciclos, ordenamiento topológico, componentes conexas	Camino más corto, niveles de árbol, grafo bipartito

Cuadro 1: Comparación entre DFS y BFS

2.5. Ordenamiento topológico (Topological Sort)

El ordenamiento topológico es un ordenamiento lineal de los vértices en un grafo dirigido acíclico (DAG) tal que para cada arista dirigida (u, v), el vértice u viene antes del vértice v en el ordenamiento. En otras palabras, el ordenamiento topológico respeta las dependencias entre los vértices.

Nota importante: El ordenamiento topológico solo existe para grafos dirigidos acíclicos (DAGs). Si el grafo contiene un ciclo, no es posible ordenar los vértices de manera que se respeten todas las dependencias, ya que habría una dependencia circular. Además, un DAG puede tener múltiples ordenamientos topológicos válidos.

2.5.1. Ejemplo de DAG con Dependencias

2.5.2. Ordenamiento topológico de Tareas

Para el DAG mostrado en la Figura 16, un ordenamiento topológico válido sería:

1. A: Inicio
2. B: Diseño

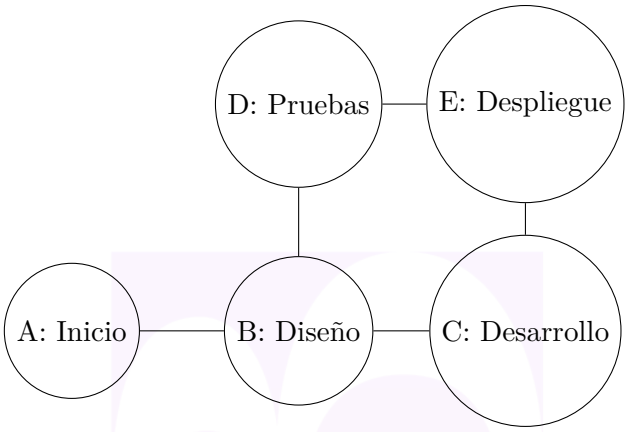


Figura 16: Ejemplo de DAG representando dependencias de tareas en un proyecto

3. C: Desarrollo
4. D: Pruebas
5. E: Despliegue

Este ordenamiento respeta todas las dependencias:

- El diseño (B) debe completarse antes del desarrollo (C) y las pruebas (D)
- Tanto el desarrollo (C) como las pruebas (D) deben completarse antes del despliegue (E)
- El inicio (A) debe ser la primera tarea

2.5.3. Aplicaciones del Ordenamiento Topológico

El ordenamiento topológico tiene varias aplicaciones prácticas, incluyendo:

- **Planificación de tareas**: Determinar el orden en que se deben realizar las tareas en un proyecto, teniendo en cuenta las dependencias entre ellas.
- **Análisis de dependencias**: Identificar las dependencias entre módulos de software o componentes de hardware.
- **Compilación de código**: Determinar el orden en que se deben compilar los archivos de código fuente, teniendo en cuenta las dependencias entre ellos.
- **Resolución de dependencias**: Resolver las dependencias entre paquetes de software o bibliotecas.

2.5.4. Pseudo-código para el Ordenamiento Topológico

Un algoritmo para ordenamiento topológico se basa en una búsqueda en profundidad (DFS) para calcular el post-orden de los vértices. El ordenamiento topológico se obtiene listando los vértices en el orden inverso del post-orden.

Algorithm 3 OrdenamientoTopológicoDFS(grafo)

```
procedure OrdenamientoTopológicoDFS(grafo)
    Crear una lista L para almacenar el ordenamiento topológico
    Crear un conjunto de vértices visitados
    for cada vértice en el grafo do
        if vértice no está visitado then
            DFS(grafo, vértice, L, visitados)
        end if
    end for
    return L
end procedure
procedure DFS(grafo, vértice, L, visitados)
    Marcar vértice como visitado
    for cada vecino del vértice en el grafo do
        if vecino no está visitado then
            DFS(grafo, vecino, L, visitados)
        end if
    end for
    Agregar vértice al inicio de L                    ▷ Añadir en post-orden
end procedure
```

Este algoritmo primero realiza una búsqueda en profundidad en el grafo. Cuando la búsqueda en profundidad completa un vértice (es decir, después de visitar todos sus vecinos), el vértice se agrega al inicio de una lista. La lista resultante, cuando se invierte, da el ordenamiento topológico.

Ejemplo : Analizar la ejecución de OrdenamientoTopológicoDFS en el grafo de la figura 17

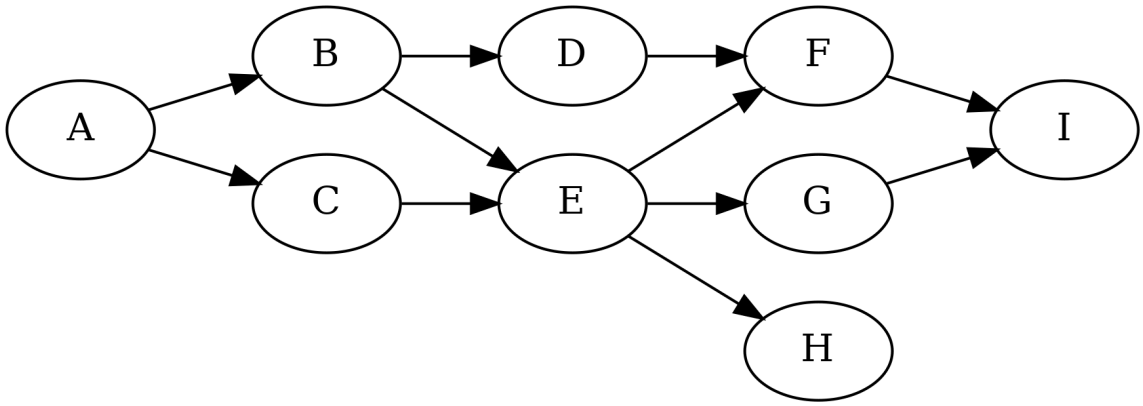


Figura 17: Ejemplo para análisis del ordenamiento topológico

Otro algoritmo común para el ordenamiento topológico es el algoritmo de Kahn, que utiliza una cola para mantener un registro de los vértices con grado de entrada cero.

2.5.5. Complejidad del Ordenamiento Topológico

La complejidad temporal del algoritmo de Kahn es $O(V+E)$, donde V es el número de vértices y E es el número de aristas en el grafo. Esto se debe a que el algoritmo visita cada vértice y cada

Algorithm 4 OrdenamientoTopológicoKahn(grafo)

```
procedure OrdenamientoTopológicoKahn(grafo)
    Crear una cola Q
    Encolar todos los vértices con grado de entrada 0 en Q
    Crear una lista L para almacenar el ordenamiento topológico
    while Q no está vacía do
        u = desencolar de Q
        Agregar u a L
        for cada vecino v de u en el grafo do
            Reducir el grado de entrada de v en 1
            if el grado de entrada de v es 0 then
                Encolar v en Q
            end if
        end for
    end while
    if el grafo tiene aristas then
        Imprimir .Error: el grafo tiene un ciclo"
    else
        Devolver L
    end if
end procedure
```

arista una vez. La complejidad espacial del algoritmo es $O(V)$, ya que en el peor de los casos, la cola puede contener todos los vértices del grafo.

3. Caminos más cortos (Shortest paths)

3.1. Grafos con Peso o Costo

En un *grafo con peso* (también llamado grafo ponderado), cada arista tiene asociado un valor numérico, que representa su *peso* o *costo*. Este peso puede representar diferentes magnitudes dependiendo de la aplicación, como por ejemplo:

- Distancia física entre dos puntos (en kilómetros, metros, etc.).
- Costo monetario de viajar entre dos ciudades.
- Tiempo necesario para ir de un lugar a otro.
- Ancho de banda disponible en una conexión de red.

Formalmente, un grafo con peso se define como $G = (V, E, w)$, donde V es el conjunto de vértices, E es el conjunto de aristas, y $w : E \to \mathbb{R}$ es una función que asigna un peso a cada arista.

3.2. El Problema del Camino Más Corto

El *problema del camino más corto* consiste en encontrar el camino entre dos vértices de un grafo, tal que la suma de los pesos de las aristas que lo componen sea mínima. Este problema tiene muchas aplicaciones prácticas, entre las que se destacan:

- **Navegación GPS:** Encontrar la ruta más rápida entre dos ubicaciones en un mapa.
- **Planificación de rutas:** Determinar la secuencia óptima de entregas para un camión de reparto.
- **Redes de comunicación:** Encontrar la ruta con menor latencia para enviar datos entre dos nodos.
- **Logística:** Optimizar el flujo de materiales en una cadena de suministro.

3.3. Algoritmo de Dijkstra

El *algoritmo de Dijkstra* es un algoritmo eficiente para resolver el problema del camino más corto desde un vértice origen hacia todos los demás vértices en grafos con pesos no negativos en las aristas. **Importante:** El algoritmo no funciona correctamente si alguna arista tiene un peso negativo.

La idea principal del algoritmo es mantener un conjunto de vértices ya visitados, y para cada vértice, la distancia más corta conocida desde el vértice de origen. Inicialmente, la distancia al vértice de origen es 0, y la distancia a todos los demás vértices es infinito (∞).

El algoritmo itera seleccionando el vértice no visitado con la menor distancia conocida (utilizando una cola de prioridad), marca ese vértice como visitado, y actualiza las distancias a sus vecinos si se encuentra un camino más corto. Este proceso, llamado *relajación* de aristas, se repite hasta que todos los vértices hayan sido visitados.

3.3.1. Pseudo-código del Algoritmo de Dijkstra

Algorithm 5 Dijkstra(grafo, origen)

```
procedure DIJKSTRA( grafo, origen)
    for cada vértice v en grafo do
        distancia[v] ← ∞
        predecesor[v] ← nulo
    end for
    distancia[origen] ← 0
    Crear una cola de prioridad Q con todos los vértices, usando ́distanciaćomo clave.
    while Q no está vacía do
        u ← extraer el vértice con la menor distancia de Q
        for cada vecino v de u do
            alt ← distancia[u] + peso(u, v)
            if alt <distancia[v] then
                distancia[v] ← alt
                predecesor[v] ← u
                Actualizar la prioridad de v en Q (decrease-key)
            end if
        end for
    end while
    return distancia, predecesor
end procedure
```

3.3.2. Ejemplo de Ejecución del Algoritmo de Dijkstra

Considerara el grafo de la figura 18. Analizar el proceso de ejecución de Dijsktra y obtener el camino mas corto de A hacia los demás vértices.

3.4. Eficiencia del algoritmo de Dijkstra

La eficiencia del algoritmo de Dijkstra depende de la implementación utilizada para la cola de prioridad (priority queue). La cola de prioridad se utiliza para seleccionar el vértice no visitado con la menor distancia conocida.

- **Arreglo (Array):** La implementación más simple utiliza un arreglo para almacenar las distancias a todos los vértices. En cada iteración, se busca el vértice con la menor distancia en el arreglo, lo que toma $O(V)$ tiempo, donde V es el número de vértices. La actualización de las distancias toma $O(1)$ tiempo. Por lo tanto, la complejidad total es $O(V^2+E) = O(V^2)$, donde E es el número de aristas.

- **Lista enlazada (Linked List):** Similar al arreglo, la búsqueda del vértice con la menor distancia toma $O(V)$ tiempo. La actualización de las distancias toma $O(1)$ tiempo. La complejidad total es $O(V^2)$.

- **Montículo Binario (Binary Heap):** Un montículo binario es una estructura de datos que permite encontrar el elemento mínimo en $O(1)$ tiempo y eliminarlo en $O(\log V)$ tiempo. La actualización de las distancias toma $O(\log V)$ tiempo. Por lo tanto, la complejidad total es $O((V + E)\log V)$. En grafos densos, donde $E \approx V^2$, la complejidad se acerca a $O(V^2 \log V)$.

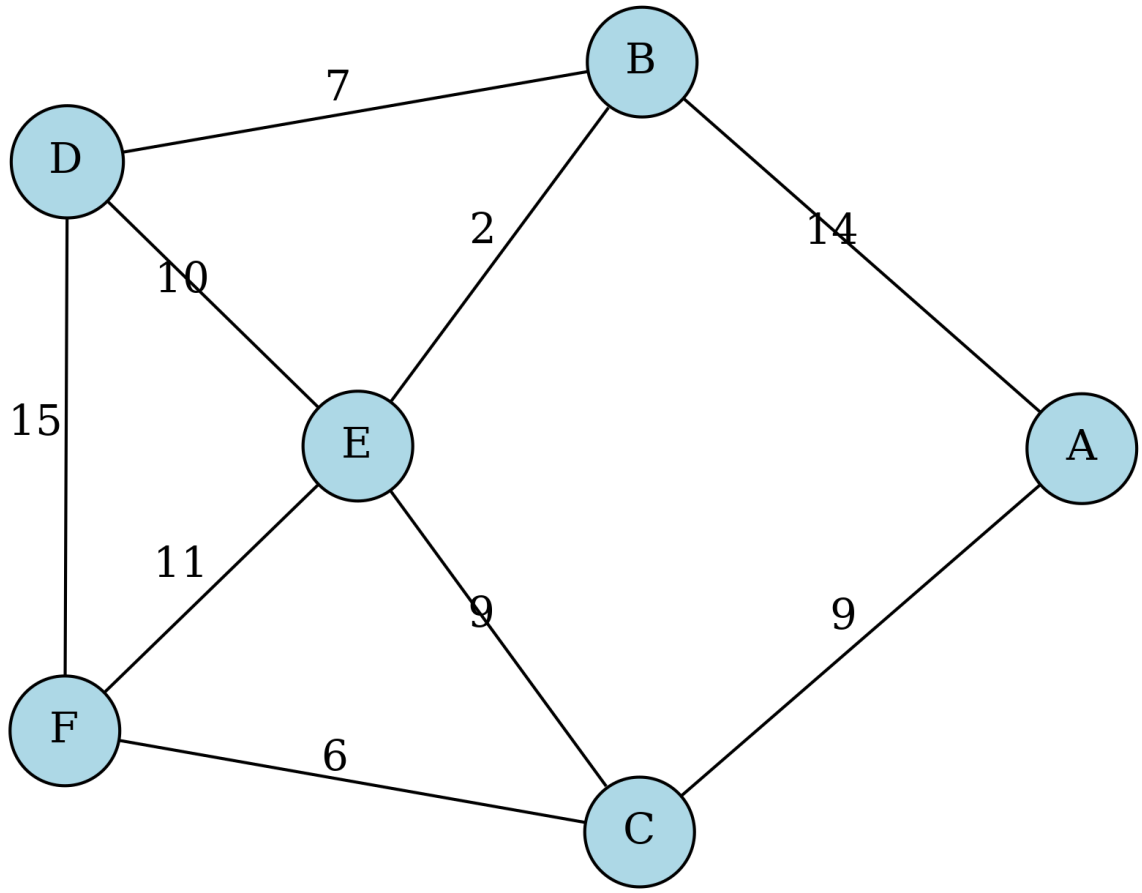


Figura 18: Grafo ponderado para buscar caminos más cortos

- **Montículo de Fibonacci (Fibonacci Heap):** Un montículo de Fibonacci es una estructura de datos más avanzada que permite encontrar el elemento mínimo en $O(1)$ tiempo (amortizado) y eliminarlo en $O(\log V)$ tiempo (amortizado). La actualización de las distancias toma $O(1)$ tiempo (amortizado). Por lo tanto, la complejidad total es $O(V \log V + E)$. Esta es la implementación más eficiente para grafos grandes y dispersos.

En resumen, la complejidad del algoritmo de Dijkstra depende de la implementación de la cola de prioridad:

- Arreglo: $O(V^2)$
- Lista enlazada: $O(V^2)$
- Montículo Binario: $O((V + E) \log V)$
- Montículo de Fibonacci: $O(V \log V + E)$

La elección de la implementación de la cola de prioridad depende del tamaño y la densidad del grafo. Para grafos pequeños, un arreglo o una lista enlazada pueden ser suficientes. Para grafos grandes y dispersos, un montículo de Fibonacci es la mejor opción. Para grafos grandes y densos, un montículo binario puede ser una buena opción.

3.5. Algoritmo de Bellman-Ford

A diferencia del algoritmo de Dijkstra, el *algoritmo de Bellman-Ford* puede encontrar el camino más corto desde un único vértice origen hacia todos los demás vértices en un grafo dirigido ponderado, incluso si algunas de las aristas tienen pesos negativos. Su capacidad para manejar pesos negativos y detectar ciclos negativos lo hace más versátil que Dijkstra, aunque generalmente es más lento con una complejidad de $O(|V||E|)$ en lugar de $O((|V| + |E|) \log |V|)$.

La idea principal del algoritmo es simple: relaja continuamente las estimaciones de la distancia a cada vértice. El algoritmo relaja todas las aristas del grafo $|V| - 1$ veces. Después de i iteraciones, se garantiza que el algoritmo ha encontrado el camino más corto que no tiene más de i aristas. Como un camino simple no puede tener más de $|V| - 1$ aristas, este número de iteraciones es suficiente para encontrar todos los caminos más cortos.

3.5.1. Detección de Ciclos Negativos

Una característica clave del algoritmo de Bellman-Ford es su capacidad para detectar ciclos de peso negativo alcanzables desde el vértice origen. Si después de $|V| - 1$ iteraciones, una $|V|$-ésima iteración todavía puede relajar alguna arista, significa que existe un ciclo de peso negativo en el grafo. Esto se debe a que la distancia a un vértice en un ciclo de este tipo podría disminuir indefinidamente.

3.5.2. Pseudo-código del Algoritmo de Bellman-Ford

Algorithm 6 BellmanFord(grafo, origen)

```
procedure BellmanFord( grafo, origen)
    for cada vértice v en grafo do
        distancia[v] ← ∞
        predecesor[v] ← nulo
    end for
    distancia[origen] ← 0
    for i de 1 a |V|-1 do
        for cada arista (u, v) con peso w en grafo do
            if distancia[u] + w <distancia[v] then
                distancia[v] ← distancia[u] + w
                predecesor[v] ← u
            end if
        end for
    end for
    for cada arista (u, v) con peso w en grafo do          ▷ Detección de ciclos negativos
        if distancia[u] + w <distancia[v] then
            return .El grafo contiene un ciclo de peso negativo"
        end if
    end for
    return distancia, predecesor
end procedure
```

3.5.3. Complejidad de Bellman-Ford

La complejidad temporal del algoritmo de Bellman-Ford es de $O(|V||E|)$, ya que el algoritmo consiste en dos bucles anidados: el bucle exterior se ejecuta $|V| - 1$ veces y el interior itera sobre todas las aristas $|E|$. El bucle final para la detección de ciclos negativos toma $O(|E|)$ tiempo, por lo que no cambia la complejidad general.

3.6. Algoritmo de Floyd-Warshall

El *algoritmo de Floyd-Warshall* resuelve el problema de los caminos más cortos para **todos los pares de vértices** en un grafo dirigido y ponderado. A diferencia de Dijkstra y Bellman-Ford que calculan caminos desde un único origen, Floyd-Warshall calcula la matriz completa de distancias mínimas entre todos los pares de vértices. Al igual que Bellman-Ford, puede manejar aristas con pesos negativos, pero no puede manejar ciclos de peso negativo (si existe un ciclo de peso negativo, la distancia del camino más corto no está bien definida pues puede decrecer indefinidamente).

Este algoritmo es un excelente ejemplo de *programación dinámica*. La idea es considerar progresivamente vértices intermedios en los caminos. El algoritmo calcula los caminos más cortos entre todos los pares de vértices (i, j) utilizando un conjunto creciente de vértices intermedios $\{1, 2, \ldots, k\}$, preguntándose en cada paso: "¿Es más corto el camino de i a j pasando por k que el camino directo conocido hasta ahora?"

3.6.1. Pseudo-código del Algoritmo de Floyd-Warshall

Algorithm 7 FloydWarshall(grafo)

```
procedure FloydWarshall(grafo)
    Sea dist una matriz de |V|x|V| de distancias mínimas.
    for cada vértice v en grafo do
        for cada vértice u en grafo do
            if u == v then
                dist[u][v] ← 0
            else if existe arista (u,v) then
                dist[u][v] ← peso(u,v)
            else
                dist[u][v] ← ∞
            end if
        end for
    end for
    for k de 1 a |V| do
        for i de 1 a |V| do
            for j de 1 a |V| do
                if dist[i][j] >dist[i][k] + dist[k][j] then
                    dist[i][j] ← dist[i][k] + dist[k][j]
                end if
            end for
        end for
    end for
    return dist
end procedure
```

3.6.2. Complejidad de Floyd-Warshall

La complejidad temporal del algoritmo de Floyd-Warshall es de $O(|V|^3)$, ya que el algoritmo consiste en tres bucles anidados que iteran sobre todos los vértices. La complejidad espacial es de $O(|V|^2)$ para almacenar la matriz de distancias.

3.7. Comparación de Algoritmos de Caminos Más Cortos

La siguiente tabla resume las características principales de los tres algoritmos de caminos más cortos estudiados:

Guía de selección:

- Usa **Dijkstra** cuando necesites caminos desde un origen y todos los pesos sean no negativos (caso más común).
- Usa **Bellman-Ford** cuando necesites detectar ciclos negativos o cuando haya aristas con pesos negativos.
- Usa **Floyd-Warshall** cuando necesites las distancias entre todos los pares de vértices y el grafo sea pequeño ($|V| < 500$ aproximadamente).

Algoritmo	**Desde**	**Pesos Negativos**	**Complejidad**	**Uso Principal**
Dijkstra	Un origen a todos	No	$O((V+E)\log V)$ con heap binario	Caminos desde un origen en grafos con pesos ≥ 0
Bellman-Ford	Un origen a todos	Sí	$O(VE)$	Caminos desde un origen con pesos negativos
Floyd-Warshall	Todos a todos	Sí	$O(V^3)$	Caminos entre todos los pares de vértices

Cuadro 2: Comparación de algoritmos de caminos más cortos

3.8. Algoritmo A*

El algoritmo A* (pronunciado .A estrella") es una extensión del algoritmo de Dijkstra que mejora el rendimiento mediante el uso de una función heurística para guiar la búsqueda. Mientras que Dijkstra explora los caminos en orden de su costo actual desde el nodo de inicio (sin considerar hacia dónde se dirige), A* utiliza una función de evaluación, $f(n)$, que estima el costo total del camino más corto desde el inicio hasta el destino que pasa por el nodo n.

La ventaja de A* sobre Dijkstra es que puede explorar significativamente menos nodos al dirigir la búsqueda hacia el destino, lo que resulta en un mejor rendimiento práctico, especialmente en grafos grandes como mapas de navegación.

La función de evaluación se define como: $f(n) = g(n) + h(n)$ donde:

- $g(n)$ es el costo real del camino desde el nodo de inicio hasta el nodo n.
- $h(n)$ es una estimación heurística del costo desde el nodo n hasta el nodo de destino.

La clave del rendimiento de A* radica en la calidad de la función heurística $h(n)$.

3.8.1. Función Heurística

Una función heurística es una "suposición informada"que estima el costo restante para llegar al destino. Para que A* garantice encontrar el camino más corto, la heurística debe cumplir ciertas propiedades:

- **Admisibilidad**: Una heurística es admisible si nunca sobrestima el costo real para llegar al destino. Es decir, $h(n)$ siempre debe ser menor o igual que el costo real del camino más corto desde n hasta el destino.
- **Consistencia** (o monotonicidad): Una heurística es consistente si, para cada nodo n y cada sucesor p de n, el costo estimado de llegar al destino desde n no es mayor que el costo de llegar a p más el costo estimado de llegar al destino desde p. Formalmente: $h(n) \leq c(n, p) + h(p)$, donde $c(n, p)$ es el costo del arco de n a p.

Si la heurística es admisible, A* garantiza encontrar el camino óptimo (más corto). Si además es consistente, el algoritmo será más eficiente ya que no necesitará re-evaluar nodos ya visitados, pues se garantiza que la primera vez que un nodo es expandido, ya se ha encontrado el camino más corto hacia él.

Ejemplos de heurísticas comunes:

- **Distancia Euclidiana**: $h(n) = \sqrt{(x_n - x_{destino})^2 + (y_n - y_{destino})^2}$. Útil para movimiento en cualquier dirección (como un mapa).
- **Distancia Manhattan**: $h(n) = |x_n - x_{destino}| + |y_n - y_{destino}|$. Útil cuando solo se permite movimiento horizontal y vertical (como en una cuadrícula).
- **Distancia de Chebyshev**: $h(n) = \text{máx}(|x_n - x_{destino}|, |y_n - y_{destino}|)$. Útil cuando se permiten movimientos diagonales.

Todas estas heurísticas son admisibles porque nunca sobrestiman la distancia real al destino.

3.8.2. Pseudo-código del Algoritmo A*

Algorithm 8 A*(inicio, destino)

```
procedure AStar(inicio, destino)
    Crear una lista abierta (open list) para los nodos por visitar, inicializada con el nodo de inicio.
    Crear una lista cerrada (closed list) para los nodos ya visitados, inicialmente vacía.
    g[inicio] ← 0
    f[inicio] ← h(inicio)
    while la lista abierta no está vacía do
        n ← el nodo en la lista abierta con el menor valor de f.
        if n es el nodo destino then
            return reconstruir el camino desde el destino hasta el inicio.
        end if
        Mover n de la lista abierta a la lista cerrada.
        for cada vecino m de n do
            if m está en la lista cerrada then
                continue
            end if
            g_tentativo ← g[n] + costo(n, m)
            if m no está en la lista abierta then
                Añadir m a la lista abierta.
            else if g_tentativo ≥ g[m] then
                continue                                  ▷ Este no es un camino mejor.
            end if
            g[m] ← g_tentativo
            f[m] ← g[m] + h(m)
            Guardar n como el predecesor de m.
        end for
    end while
    return fallo (no se encontró un camino).
end procedure
```

3.8.3. Ejemplo de Ejecución del Algoritmo A*

Consideremos el siguiente grafo, donde el objetivo es encontrar el camino más corto desde el nodo A hasta el nodo F. Los pesos de las aristas son los costos $g(n)$, y los valores heurísticos $h(n)$ para cada nodo se muestran en la tabla.

El algoritmo se ejecutaría de la siguiente manera:

1. **Inicialización**:
   - Lista Abierta: {A}
   - $g(A) = 0$, $f(A) = g(A) + h(A) = 0 + 7 = 7$
2. **Iteración 1**:
   - Sacar A de la lista abierta. Vecinos: B, C.

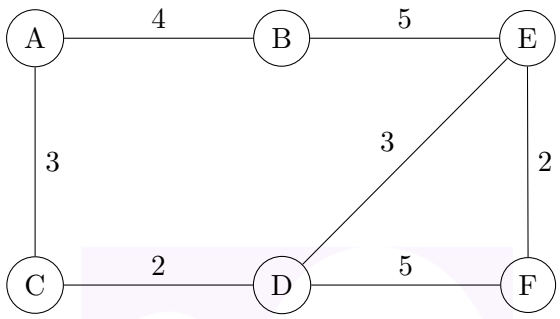


Figura 19: Grafo para el ejemplo de A*

Nodo	**h(n)**
A	7
B	6
C	5
D	3
E	2
F	0

Cuadro 3: Valores heurísticos para cada nodo

- Evaluar B: $g(B) = 4,\ f(B) = 4 + 6 = 10$.
- Evaluar C: $g(C) = 3,\ f(C) = 3 + 5 = 8$.
- Lista Abierta: {C(8), B(10)}

3. **Iteración 2**:
   - Sacar C (menor f) de la lista abierta. Vecinos: D.
   - Evaluar D: $g(D) = g(C) + 2 = 3 + 2 = 5,\ f(D) = 5 + 3 = 8$.
   - Lista Abierta: {D(8), B(10)}
4. **Iteración 3**:
   - Sacar D (menor f) de la lista abierta. Vecinos: E, F.
   - Evaluar E: $g(E) = g(D) + 3 = 5 + 3 = 8,\ f(E) = 8 + 2 = 10$.
   - Evaluar F: $g(F) = g(D) + 5 = 5 + 5 = 10,\ f(F) = 10 + 0 = 10$.
   - Lista Abierta: {B(10), E(10), F(10)}
5. **Iteración 4**:
   - Sacar B (se puede elegir cualquiera de los nodos con f=10). Vecinos: E.
   - Evaluar E: $g(E) = g(B) + 5 = 4 + 5 = 9$. Como el costo actual para llegar a E (8) es menor, no se actualiza.
   - Lista Abierta: {E(10), F(10)}
6. **Iteración 5**:
   - Sacar E. Vecinos: F.

- Evaluar F: $g(F) = g(E) + 2 = 8 + 2 = 10$. El costo es el mismo, no se actualiza.
- Lista Abierta: {F(10)}

7. **Iteración 6**:

   - Sacar F. Es el destino. Algoritmo termina.

Reconstruyendo el camino desde F usando los predecesores guardados, el camino más corto encontrado es: **A** $\rightarrow$ **C** $\rightarrow$ **D** $\rightarrow$ **F** con un costo total de **10**.

3.8.4. Complejidad y Optimalidad

La complejidad de A* depende de la heurística. En el peor de los casos, con una heurística no informativa ($h(n) = 0$ para todo n), A* se comporta como Dijkstra, y su complejidad es $O(E + V \log V)$ con una cola de prioridad eficiente. Con una heurística perfecta, la complejidad es lineal en la longitud del camino.

A* es óptimo y completo, lo que significa que siempre encontrará el camino más corto si existe, siempre que la heurística sea admisible.

4. Árboles de Cubrimiento Mínimo (Minimum Spanning Trees)

Un *árbol de expansión mínima* (Minimum Spanning Tree (MST)) de un grafo conexo no dirigido con pesos en las aristas es un subgrafo que cumple las siguientes propiedades:

- Es un árbol (grafo conexo y acíclico)
- Conecta todos los vértices del grafo original
- La suma de los pesos de sus aristas es la mínima posible entre todos los árboles de expansión

Por definición, un árbol con $|V|$ vértices tiene exactamente $|V|-1$ aristas. Por lo tanto, un MST de un grafo con $|V|$ vértices contiene exactamente $|V|-1$ aristas.

Las aplicaciones de los MST incluyen:

- **Diseño de redes:** Conectar un conjunto de ubicaciones con la menor cantidad de cableado posible.
- **Clustering:** Agrupar puntos de datos en función de su proximidad.
- **Análisis de circuitos:** Encontrar la configuración de componentes que minimice el costo total.
- **Diseño de rutas de transporte:** Optimizar la conexión de ciudades o nodos con el menor costo de infraestructura.

4.1. Algoritmo de Kruskal

El *algoritmo de Kruskal* es un algoritmo *voraz* (greedy) para encontrar el MST de un grafo conexo no dirigido con pesos en las aristas. La idea principal del algoritmo es construir el MST agregando aristas de forma incremental, siempre eligiendo la arista de menor peso que no forme un ciclo con las aristas ya seleccionadas.

La estrategia es simple:

1. Ordenar todas las aristas por peso (de menor a mayor)
2. Ir agregando aristas en ese orden, siempre que no formen un ciclo
3. Detenerse cuando se hayan agregado $|V|-1$ aristas

Para detectar eficientemente si agregar una arista formaría un ciclo, el algoritmo utiliza una estructura de datos de *conjuntos disjuntos* (Union-Find o Disjoint Set Union).

4.1.1. Pseudo-código del Algoritmo de Kruskal

Algorithm 9 Kruskal(grafo)

```
procedure KRUSKAL(grafo)
    Crear un conjunto de aristas del MST vacío
    Ordenar todas las aristas del grafo en orden ascendente de peso
    Crear una estructura Union-Find para cada vértice
    for cada arista (u, v) con peso w en el orden ascendente do
        if Find(u) ≠ Find(v) then
            Agregar (u, v) al conjunto de aristas del MST
            Union(u, v)
        end if
    end for
    return conjunto de aristas del MST
end procedure
```

4.1.2. Complejidad del Algoritmo de Kruskal

La complejidad temporal del algoritmo de Kruskal es $O(|E| \log |E|)$ o $O(|E| \log |V|)$, donde $|E|$ es el número de aristas y $|V|$ es el número de vértices. Esto se debe principalmente al tiempo de ordenamiento de las aristas. Las operaciones de Union-Find tienen una complejidad casi constante en promedio, especialmente con optimizaciones como compresión de caminos y unión por rango/tamaño.

4.2. Algoritmo de Prim

El *algoritmo de Prim* es otro algoritmo *voraz* (greedy) para encontrar el MST de un grafo conexo no dirigido con pesos en las aristas. A diferencia de Kruskal que considera aristas globalmente, Prim construye el MST de forma incremental manteniendo siempre un árbol conectado.

La estrategia es la siguiente:

1. Comenzar con un vértice arbitrario (este forma un árbol de un solo nodo)
2. En cada iteración, agregar la arista de menor peso que conecta un vértice ya en el árbol con un vértice fuera del árbol
3. Repetir hasta que todos los vértices estén en el árbol

Prim es similar a Dijkstra en su estructura, pero en lugar de minimizar la distancia total desde el origen, minimiza el peso de la arista que conecta cada nuevo vértice al árbol.

4.2.1. Pseudo-código del Algoritmo de Prim

Algorithm 10 Prim(grafo, origen)

```
procedure PRIM(grafo, origen)
    Crear un conjunto de vértices en el MST vacío
    Crear una cola de prioridad (min-heap) para almacenar aristas
    Agregar (0, origen) a la cola de prioridad (peso, vértice)
    Inicialización
    for cada vértice v en grafo do
        clave[v] ← infinito
        padre[v] ← nulo
    end for
    clave[origen] ← 0
    Iteración principal
    while la cola de prioridad no está vacía do
        u ← extraer vértice con la menor clave de la cola de prioridad
        Agregar u al conjunto de vértices en el MST
        for cada vecino v de u do
            if v no está en el MST y peso(u, v) <clave[v] then
                clave[v] ← peso(u, v)
                padre[v] ← u
                Actualizar v en la cola de prioridad con la nueva clave
            end if
        end for
    end while
    return padre (representa el MST)
end procedure
```

4.2.2. Complejidad del Algoritmo de Prim

La complejidad temporal del algoritmo de Prim es $O((|V| + |E|) \log |V|)$ o $O(|E| \log |V|)$ si se utiliza una cola de prioridad basada en montículo binario. Si se utiliza un montículo de Fibonacci, la complejidad puede reducirse a $O(|E| + |V| \log |V|)$.

4.3. Comparación de Algoritmos de MST

La siguiente tabla compara los dos algoritmos principales para encontrar el MST:

Algoritmo	**Estrategia**	**Estructura de Datos**	**Complejidad**
Kruskal	Selecciona aristas globalmente por peso	Union-Find	$O(E \log E)$ o $O(E \log V)$
Prim	Expande árbol desde un vértice inicial	Cola de prioridad (heap binario)	$O((V + E) \log V)$ con heap binario

Cuadro 4: Comparación de algoritmos de MST

Guía de selección:

- Usa **Kruskal** para grafos dispersos (donde $|E|$ es cercano a $|V|$) o cuando las aristas ya están ordenadas o pueden ordenarse eficientemente.
- Usa **Prim** para grafos densos (donde $|E|$ es cercano a $|V|^2$) o cuando se tiene una representación de lista de adyacencias eficiente.

Ambos algoritmos son correctos y producen un MST. La elección entre ellos depende principalmente de la densidad del grafo y la representación de datos disponible.

5. Flujo Máximo / Corte mínimo

5.1. Flujo Máximo (Maximum Flow)

El *problema del flujo máximo* consiste en encontrar la cantidad máxima de "flujo"que puede pasar desde un vértice fuente (origen) a un vértice sumidero (destino) en una red de flujo. Una red de flujo es un grafo dirigido donde cada arista tiene una *capacidad*, que representa la cantidad máxima de flujo que puede pasar a través de ella.

Un flujo válido debe satisfacer dos restricciones:

- **Restricción de capacidad:** El flujo a través de cada arista no puede exceder su capacidad: $f(u, v) \leq c(u, v)$ para toda arista (u, v).
- **Conservación del flujo:** Para todo vértice excepto la fuente y el sumidero, el flujo total que entra debe ser igual al flujo total que sale: $\sum_u f(u, v) = \sum_w f(v, w)$ para todo vértice v que no sea fuente ni sumidero.

Las aplicaciones del flujo máximo incluyen:

- **Optimización de redes de distribución:** Maximizar el flujo de bienes, agua, electricidad o información a través de una red.
- **Asignación de recursos:** Asignar tareas a trabajadores o recursos a proyectos para maximizar la eficiencia o la producción.
- **Análisis de tráfico:** Determinar la capacidad máxima de una red de carreteras o de transporte público.
- **Planificación de evacuaciones:** Calcular la cantidad máxima de personas que pueden ser evacuadas de un área en un tiempo dado.

5.2. Algoritmo de Ford-Fulkerson

El *algoritmo de Ford-Fulkerson* es un algoritmo fundamental para encontrar el flujo máximo en una red de flujo. La idea principal del algoritmo es encontrar y utilizar repetidamente çaminos aumentantes"(augmenting paths) para incrementar el flujo. Para ello, se utiliza el concepto de *grafo residual*.

El *grafo residual* G_f es un grafo que representa la capacidad de flujo adicional disponible en la red en cada momento. Para cada arista (u, v) con capacidad original $c(u, v)$ y flujo actual $f(u, v)$, el grafo residual contiene:

- Una **arista directa** (u, v) con capacidad residual $c_f(u, v) = c(u, v) - f(u, v)$ si $f(u, v) < c(u, v)$. Esta representa cuánto flujo adicional puede pasar por la arista original.
- Una **arista inversa** (v, u) con capacidad residual $c_f(v, u) = f(u, v)$ si $f(u, v) > 0$. Esta arista inversa permite "deshacer.º redirigir el flujo si es necesario para encontrar un mejor camino aumentante.

Un *camino aumentante* es cualquier camino desde la fuente al sumidero en el grafo residual. El flujo puede incrementarse en la cantidad igual a la capacidad residual mínima (cuello de botella) a lo largo de ese camino.

5.2.1. Pseudo-código del Algoritmo de Ford-Fulkerson

Algorithm 11 FordFulkerson(grafo, fuente, sumidero)

```
procedure FordFulkerson(grafo, fuente, sumidero)
    Inicializar el flujo f(u, v) en todas las aristas a 0
    flujo_maximo ← 0
    while exista un camino aumentante P desde la fuente al sumidero en el grafo residual G_f do
        Encontrar un camino aumentante P (por ejemplo, usando BFS o DFS)
        Calcular la capacidad residual mínima (bottleneck capacity) c_f(P) del camino P
        flujo_maximo ← flujo_maximo + c_f(P)
        for cada arista (u, v) en P do
            Aumentar el flujo en (u, v) por c_f(P): f(u, v) ← f(u, v) + c_f(P)
            Disminuir el flujo en la arista inversa (v, u) por c_f(P): f(v, u) ← f(v, u) − c_f(P)
        end for
    end while
    return flujo_maximo
end procedure
```

5.2.2. Complejidad del Algoritmo de Ford-Fulkerson

La complejidad del algoritmo de Ford-Fulkerson depende de cómo se encuentre el camino aumentante. En el peor de los casos, la complejidad puede ser $O(|E|f^*)$, donde f^* es el valor del flujo máximo. Si las capacidades son enteras, el algoritmo termina. Sin embargo, si las capacidades son números reales, el algoritmo puede no converger o converger a un valor incorrecto.

Existen versiones mejoradas del algoritmo de Ford-Fulkerson que garantizan una complejidad polinomial:

- **Algoritmo de Edmonds-Karp:** Utiliza BFS para encontrar el camino aumentante más corto en el grafo residual. Su complejidad es $O(|V||E|^2)$.
- **Algoritmo de Dinic:** Una mejora del algoritmo de Edmonds-Karp que utiliza un grafo de niveles y bloquea el flujo a lo largo de múltiples caminos. Su complejidad es $O(|V|^2|E|)$ y $O(|V|^{2/3}|E|)$ o $O(\text{mín}(|V|^{2/3}, |E|^{1/2})|E|)$ para grafos especiales.

5.3. Corte Mínimo (Minimum Cut)

El *problema del corte mínimo* está estrechamente relacionado con el problema del flujo máximo. Un *corte* en una red de flujo es una partición de los vértices en dos conjuntos disjuntos, S y T, tal que la fuente s pertenece a S y el sumidero t pertenece a T.

La *capacidad de un corte* (S, T) es la suma de las capacidades de las aristas que van del conjunto S al conjunto T:

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

El *corte mínimo* es el corte con la menor capacidad entre todos los cortes posibles que separan la fuente del sumidero.

5.3.1. Teorema de Flujo Máximo - Corte Mínimo

El *teorema de flujo máximo-corte mínimo* (Max-Flow Min-Cut Theorem) es uno de los resultados más importantes en la teoría de redes. Establece que:

El valor del flujo máximo en una red es igual a la capacidad del corte mínimo en esa red.

Formalmente: $\text{máx}\, f = \text{mín}\, c(S, T)$, donde f es el valor del flujo y $c(S, T)$ es la capacidad de un corte.

Este teorema proporciona una poderosa dualidad entre los problemas de flujo y corte:

- Si se encuentra un flujo cuyo valor es igual a la capacidad de algún corte, entonces ese flujo es máximo y ese corte es mínimo.
- El algoritmo de Ford-Fulkerson encuentra implícitamente tanto el flujo máximo como el corte mínimo.
- El corte mínimo se puede obtener al final del algoritmo identificando el conjunto de vértices alcanzables desde la fuente en el grafo residual final.

5.3.2. Aplicaciones del Corte Mínimo

Las aplicaciones del corte mínimo incluyen:

- **Segmentación de imágenes:** Separar objetos en una imagen identificando un corte de mínima capacidad en un grafo que representa los píxeles y sus relaciones.
- **Análisis de fiabilidad de redes:** Identificar los puntos débiles o cuellos de botella en una red de comunicación o infraestructura, cuya eliminación desconectaría la fuente del sumidero con el menor costo.
- **Optimización de la producción:** Determinar la capacidad máxima de un sistema de producción o la cantidad mínima de recursos necesarios para mantener una cierta producción.
- **Minería de datos y aprendizaje automático:** En problemas de clustering y clasificación, donde el corte mínimo puede ayudar a encontrar particiones óptimas de datos.